

PADARIA VILA ROMANA
Sistema Completo de Banco de
Dados com Controle de Estoque

Professor: Celso Luis Caldeira

Disciplina: Banco de Dados

Material Didático Completo

Introdução

Este projeto simula um sistema real de controle de entrada de mercadorias e estoque automático.

Criação do Banco

Explicação: Cria e ativa o banco.

```
CREATE DATABASE padaria_vila_romana;  
USE padaria_vila_romana;
```

Tabelas

```
CREATE TABLE fornecedores (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(150) NOT NULL,  
  email VARCHAR(150),  
  celular VARCHAR(20)  
);
```

```
CREATE TABLE categorias (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  descricao VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE produtos (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  descricao_produto VARCHAR(150) NOT NULL,  
  id_categoria INT,  
  preco_unitario DECIMAL(10,2),  
  FOREIGN KEY (id_categoria) REFERENCES  
categorias(id)  
);
```

```
CREATE TABLE notas_fiscais (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  id_fornecedor INT,  
  numero_nota_fiscal VARCHAR(50),  
  data DATE,  
  FOREIGN KEY (id_fornecedor) REFERENCES  
fornecedores(id)  
);
```

```
CREATE TABLE itens_notas (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  id_nota INT,  
  id_produto INT,  
  quantidade DECIMAL(10,2),
```

```
preco_unitario DECIMAL(10,2),  
  FOREIGN KEY (id_nota) REFERENCES  
notas_fiscais(id),  
  FOREIGN KEY (id_produto) REFERENCES produtos(id)  
);
```

Controle de Estoque

Explicação: Mantém saldo atualizado.

```
CREATE TABLE estoque (  
    id_produto INT PRIMARY KEY,  
    quantidade DECIMAL(10,2) DEFAULT 0,  
    FOREIGN KEY (id_produto) REFERENCES produtos(id)  
);
```

Explicação: Atualiza estoque automaticamente.

```
DELIMITER $$  
CREATE TRIGGER trg_entrada_estoque  
AFTER INSERT ON itens_notas  
FOR EACH ROW  
BEGIN  
    INSERT INTO estoque VALUES (NEW.id_produto,  
NEW.quantidade)  
    ON DUPLICATE KEY UPDATE quantidade = quantidade +  
NEW.quantidade;  
END $$  
DELIMITER ;
```

Explicação: Simula baixa de estoque.

```
DELIMITER $$  
CREATE TRIGGER trg_saida_simulada  
AFTER DELETE ON itens_notas  
FOR EACH ROW  
BEGIN  
    UPDATE estoque SET quantidade = quantidade -  
OLD.quantidade  
    WHERE id_produto = OLD.id_produto;  
END $$  
DELIMITER ;
```


Views

```
CREATE VIEW vw_estoque AS  
SELECT p.descricao_produto, e.quantidade  
FROM estoque e JOIN produtos p ON p.id =  
e.id_produto;
```


Functions

```
DELIMITER $$
CREATE FUNCTION fn_total_nota(p_id INT)
RETURNS DECIMAL(10,2)
BEGIN
    DECLARE total DECIMAL(10,2);
    SELECT SUM(quantidade*preco_unitario) INTO total
    FROM itens_notas WHERE id_nota=p_id;
    RETURN IFNULL(total,0);
END $$
DELIMITER ;
```

Procedures

```
DELIMITER $$
CREATE PROCEDURE sp_criar_nota(p_fornecedor
INT,p_num VARCHAR(50),p_data DATE)
BEGIN
    INSERT INTO notas_fiscais VALUES
    (NULL,p_fornecedor,p_num,p_data);
END $$
DELIMITER ;
```

Dados para Teste

```
INSERT INTO fornecedores (nome,email) VALUES
('Fornecedor A','a@email.com');
INSERT INTO categorias (descricao) VALUES
('Farinhas');
INSERT INTO produtos
(descricao_produto,id_categoria,preco_unitario)
VALUES ('Farinha',1,10);
CALL sp_criar_nota(1,'123',NOW());
INSERT INTO itens_notas
(id_nota,id_produto,quantidade,preco_unitario)
VALUES (1,1,5,10);
```

Consultas para Aula

```
SELECT * FROM vw_estoque;
```

```
SELECT fn_total_nota(1);
```

Desafios

1. Implementar estoque mínimo.
2. Criar alerta de reposição.
3. Criar relatório mensal.